

Avalon-MM SD Card Controller IP Core

Block Diagrams and Descriptions

IP core	<code>altera_avalon_mm_sdcard_controller</code>
Core version	1.0
Document version	1.0
Date	September 2026
Target	Intel FPGA / Quartus Prime, Platform Designer

Contents

1. System context	3
2. Internal structure	5
2.1 Why the shifter never stops	7
2.2 Why the FIFO is word-wide with a packer	7
3. Register map	8
4. Sequencer states	10
5. Where a block's bytes go	13
6. On the wire	15
6.1 One byte, at bit level	15
6.2 A command frame and its response	17
6.3 The start of a block read	17
6.4 The end of a block write	18
6.5 A block whose CRC16 does not match	18
Regenerating these figures	19
Revision history	19

1. System context

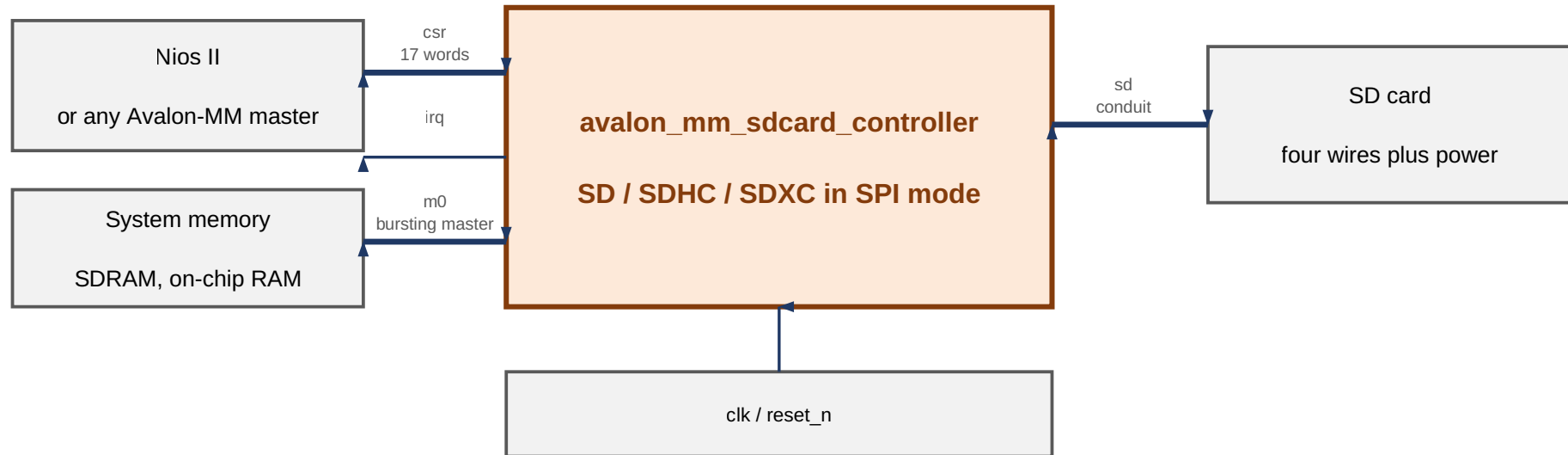
The core presents four interfaces to Platform Designer:

Interface	Type	Purpose
csr	Avalon-MM slave, 32-bit, 17 words	The register file software drives
m0	Avalon-MM master, bursting	Moves block data to and from memory. Present only when <code>USE_DMA = 1</code>
irq	Interrupt sender	One line, one mask
sd	Conduit	<code>clk</code> , <code>mosi</code> , <code>miso</code> , <code>cs_n</code> , plus <code>cd_n</code> and <code>wp_n</code> when <code>USE_CARD_DETECT = 1</code>

Two Avalon-MM interfaces rather than one is the design decision that matters most here. A controller with only a slave makes the CPU move every byte: 512 bytes is 128 loads and 128 stores per block, and at any useful SPI clock the CPU cannot look away — a missed service window starves the shifter. With `m0` the CPU writes a command and comes back when the interrupt arrives.

`m0` is still optional, because "an ordinary Avalon master would do" is a real requirement in small systems and the interface costs logic. Setting `USE_DMA` to 0 removes it from the component entirely — not just tying it off — and software moves words through the `DATA` window instead. That configuration is tested, not assumed: it is one of the five the regression sweeps, and it is the only one in which the data-phase stall timeouts can be reached at all.

System context



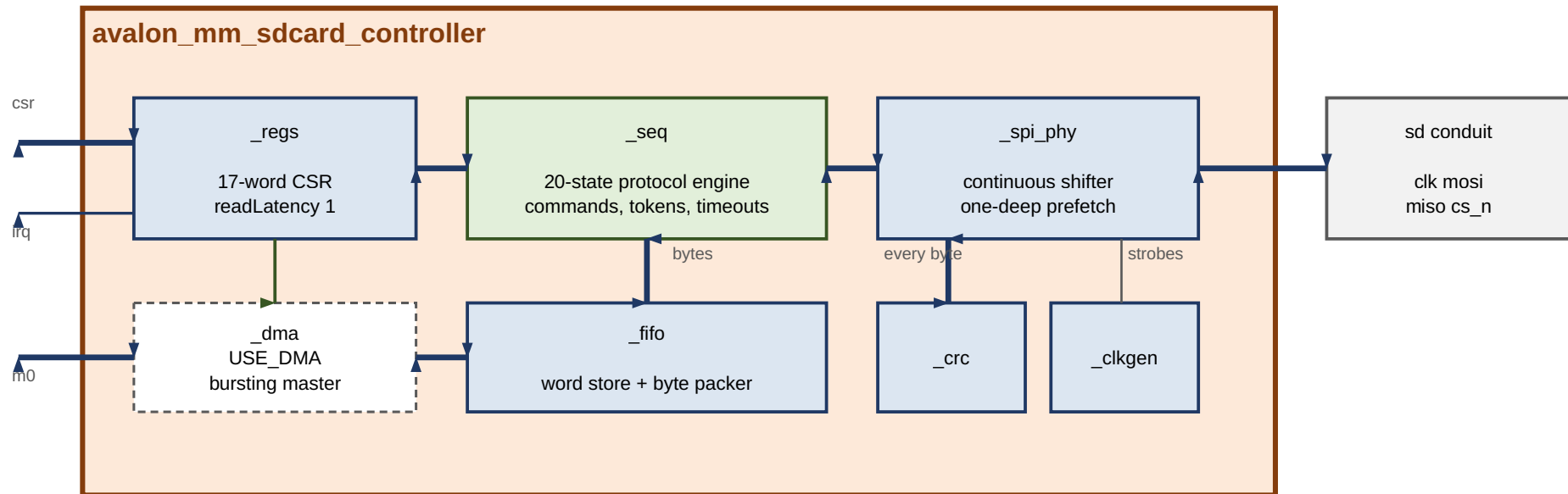
Two Avalon-MM interfaces, not one. **csr** is the slave software programs; **m0** is a master the core uses to move block data to and from memory itself, so a 512-byte block costs the CPU one command rather than 128 loads and stores. **m0** is optional - set **USE_DMA** to 0 and the interface disappears from the component, leaving software to move every word through the DATA window. The SPI conduit is four wires: **clk**, **mosi**, **miso** and **cs_n**, plus card-detect and write-protect when **USE_CARD_DETECT** is set.

2. Internal structure

Nine RTL files, 3,127 lines.

Module	What it does
<code>_pkg</code>	Register map, bit positions, protocol constants, response and phase enumerations
<code>_regs</code>	The <code>csr</code> slave. Read latency 1, one interrupt mask, write-1-to-clear status
<code>_seq</code>	The protocol engine: 20 states, command framing, token recognition, timeouts
<code>_spi_phy</code>	The shifter. Continuous, with a one-deep prefetch
<code>_clkgen</code>	Divides <code>clk</code> to the SPI rate and emits rising and falling strobes
<code>_crc</code>	CRC7 for command frames, CRC16 for data blocks, both byte-wise
<code>_fifo</code>	Word-wide store with a byte packer on the card side
<code>_dma</code>	The <code>m0</code> master. Bursts, and can abort legally

Internal structure



The shifter runs continuously while a transfer is in progress, with a one-deep prefetch, so the next byte is already queued before the current one finishes and the SPI clock never pauses between bytes. That is where the measured 98.1% of line rate comes from - 16,696 SPI clocks to move 2,048 bytes, against a theoretical 16,384. The CRC units are byte-wise rather than bit-serial for the same reason: a bit-serial CRC cannot keep up with a shifter that never stops. `_crc` computes CRC7 over command frames and CRC16 over data blocks; `_clkgen` divides the host clock to $\text{clk} / (2 \times \text{CLKDIV})$ and hands the shifter a rising and a falling strobe rather than a second clock domain.

2.1 Why the shifter never stops

The shifter runs continuously for the whole of a transfer and keeps one byte prefetched, so the next byte is queued before the current one finishes and the SPI clock does not pause between bytes. Measured, that is **16,696 SPI clocks to move 2,048 bytes** against a theoretical floor of 16,384 — **98.1% of line rate**. The regression asserts the cycle count rather than merely reporting it, because a performance number nobody checks is a number that silently regresses.

This is also why the CRC units are byte-wise rather than bit-serial. A bit-serial CRC has eight cycles to do its work only if the shifter pauses between bytes; one that does not pause needs the whole byte folded in at once.

2.2 Why the FIFO is word-wide with a packer

The two ends of the buffer run at different widths and different rates. The shifter moves one byte per eight SPI clocks; the master moves four bytes per host cycle. A byte-wide FIFO would make the master do four accesses per word, a word-wide one without a packer cannot talk to the shifter at all.

Sizing it at two blocks rather than one is what lets the DMA drain block N while the shifter is receiving block N+1. `FIFO_DEPTH_BYTES = 512` — one block — is a supported configuration and one of the five the regression sweeps, precisely because that overlap disappears and the refill path has to work mid-transfer.

3. Register map

Seventeen 32-bit words, so `CSR_ADDR_WIDTH` must be at least 5. Offsets, bit positions and reset values are given in full in the user guide, section 5.

Two things about this map are worth stating here because they are behaviours rather than layout:

Writing `CMD` with bit 31 set is what launches an operation. The write is ignored while the sequencer is busy. That is correct — a second command must not corrupt a transfer in flight — but it means software cannot assume the write took. Polling afterwards does not catch it either: busy is already clear, so the poll returns immediately for a command that never happened. Check `STATUS` first.

`ERR_INFO` records what failed, not merely that something did. It holds the last data-response token, the last R1 byte, the last data error token, and which phase the timeout occurred in. "The card never answered" and "the card answered and then never finished" need different recovery, and only the phase field distinguishes them.

Register map

Offset	Name	Access	Purpose
0x00	CTRL	RW	enable, CS control, CRC, DMA, soft resets
0x04	STATUS	RO	busy flags, FIFO level, card present, error
0x08	IRQ_ENABLE	RW	mask, one bit per source
0x0C	IRQ_STATUS	RW1C	pending events and errors
0x10	CLKDIV	RW	SPI clock = clk / (2 x CLKDIV)
0x14	TIMEOUT	RW	cycles without progress before giving up
0x18	CMD_ARG	RW	the command's 32-bit argument
0x1C	CMD	RW	index, response type, data flags; write starts
0x20	RESP0	RO	R1, or the low word of a longer response
0x24	RESP1	RO	R3 / R7 trailer
0x28	BLK_SIZE	RW	bytes per block, up to MAX_BLOCK_BYTES
0x2C	BLK_COUNT	RW	blocks in this transfer
0x30	DMA_ADDR	RW	byte address in system memory
0x34	DMA_CTRL	RW	mode; only contiguous is defined
0x38	DATA	RW	PIO window into the FIFO
0x3C	ERR_INFO	RO	last tokens, last R1, phase that failed
0x40	CORE_INFO	RO	build-time configuration, read-only

Seventeen words, so CSR_ADDR_WIDTH must be at least 5. The slave has a read latency of 1. Writing CMD with bit 31 set is what launches an operation; the write is ignored while the sequencer is busy, which is correct - a second command must not corrupt a transfer in flight - but it means software has to check STATUS first rather than assume the write took.

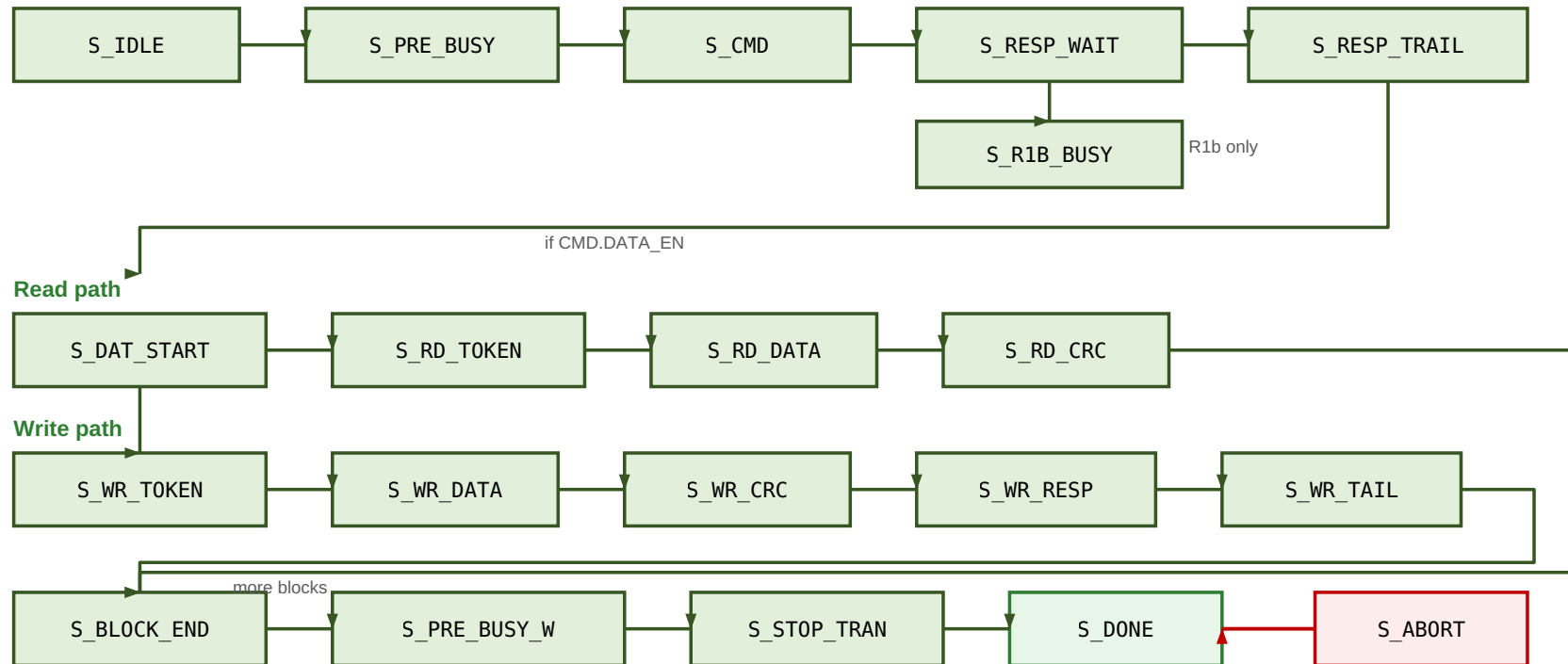
4. Sequencer states

Twenty states. The figure draws the normal path; error exits are not drawn, because every state has one and they all converge on `S_ABORT` — which is the point of having it.

What matters is where `S_ABORT` goes. Not straight to `S_IDLE`, but through `S_DONE`, so the DMA is told to abort and then allowed to drain. An Avalon master that simply stops issuing beats part-way through a burst hangs the interconnect; abandoning a transfer is more work than not starting one.

Sequencer states

Command phase



Twenty states. Error exits are not drawn - every state has one, and they all converge on **S_ABORT**, which is the point of having it. What matters is where **S_ABORT** goes: not straight to **S_IDLE** but through **S_DONE**, so the DMA is told to abort and then allowed to drain. An Avalon master that simply stops issuing beats part-way through a burst hangs the interconnect, so abandoning a transfer is more work than not starting one. **S_PRE_BUSY** free-runs 0xFF with CS high before every command, which is why the shifter is always already running when a sending state is entered - a fact the assertion suite depends on and documents.

Three states exist for reasons that are not obvious from their names:

- `S_PRE_BUSY` free-runs `0xFF` with `CS` high before every command. A card that is still programming a previous block holds `MISO` low, and clocking it is how you find out. A useful side effect: the shifter is *always already running* when a sending state is entered, which the assertion suite depends on and which `verification/check_assertions_fire.sh` documents — one fault it tried to inject turned out to be unreachable for exactly this reason.
- `S_WR_TAIL` covers the eight clocks after a data-response token during which busy on `MISO` means nothing yet. Sampling too early reads a card that has not begun as a card that has finished.
- `S_R1B_BUSY` is entered only for R1b commands, where the response is followed by the card holding the line until the operation completes.

5. Where a block's bytes go

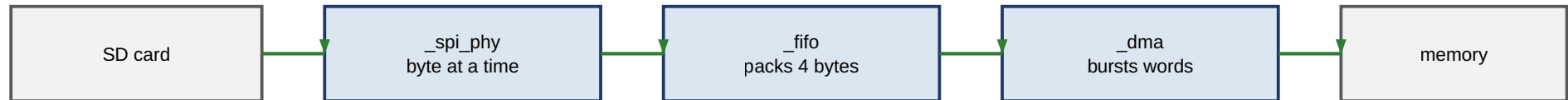
The read path and the write path are mirror images, and the third row is the same core with `USE_DMA = 0`.

In PIO mode the CPU is on a deadline. With no master keeping the buffer moving, software that is slow to service the `DATA` window starves the shifter, and the data-phase stall timeout fires. That timeout is not decoration: it was added after a configuration sweep found that `S_RD_DATA` and `S_WR_DATA` had no timeout at all, so a starved data phase hung the core until a soft reset. With a master attached the case cannot arise, which is exactly why it survived until the PIO configuration was actually run.

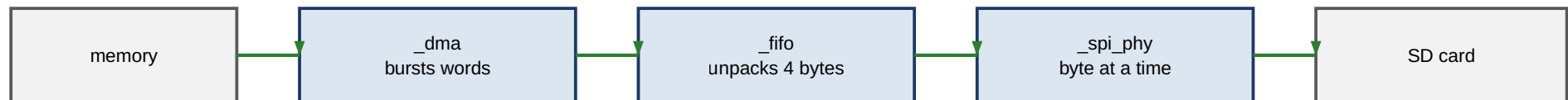
The timeout measures **time without progress**, not total duration, so a long block never trips it and a stalled one does.

Where a block's bytes go

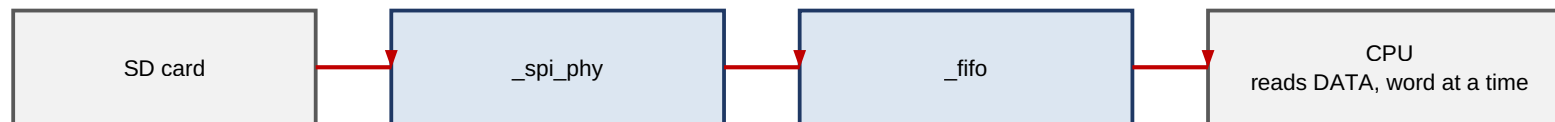
Read: card to memory



Write: memory to card



Without the DMA (`USE_DMA = 0`)

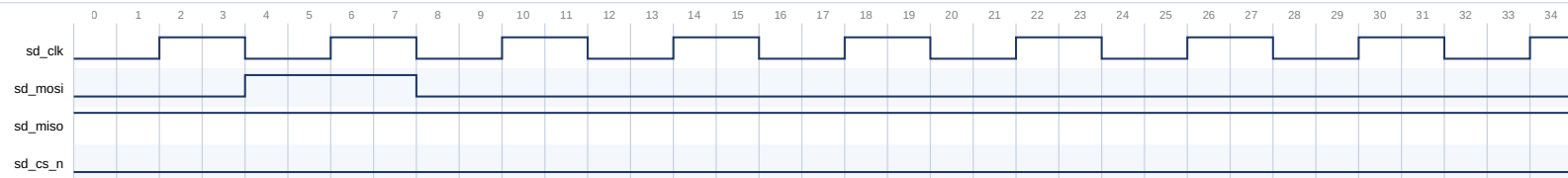


The FIFO is word-wide with a byte packer on the card side, because the two ends run at different widths and different rates: the shifter moves one byte per eight SPI clocks, the master moves four bytes per host cycle. Sizing it at two blocks rather than one is what lets the DMA drain block N while the shifter is receiving block N+1, which is where multi-block throughput comes from. In PIO mode the CPU is on a deadline: with no master to keep the buffer moving, software that is slow to service the DATA window starves the shifter and the stall timeout fires.

6. On the wire

These five figures are cut from a single recorded simulation. The bytes, tokens, CRC values and state names in them were produced by the RTL in this repository; nothing is illustrative.

6.1 One byte, at bit level



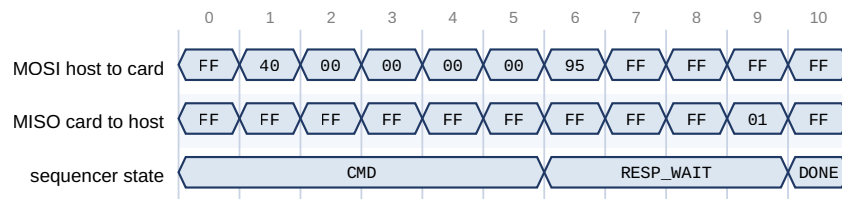
One byte on the wire: MOSI = 0x40, MISO = 0xFF, most significant bit first.

CPOL = 0, CPHA = 0. MOSI changes on the falling edge of sd_clk; both ends sample on the rising edge.

Each column is one host clock. At CLKDIV = 1 the SPI clock is clk/2, so one SPI bit is two columns.

The one thing a byte-level view cannot express, and the one an integrator has to get right: CPOL = 0, CPHA = 0. MOSI changes on the falling edge of `sd_clk` and both ends sample on the rising edge.

6.2 A command frame and its response



CMD0 GO_IDLE_STATE. Six bytes out: 0x40 | index, four argument bytes, then CRC7 shifted up one with the stop bit in bit 0.

The card answered 2 byte-times after the frame. The specification allows `N_CR` to be anything from 0 to 8, so the core polls for a byte with bit 7 clear rather than waiting a fixed time.

R1 = 0x01 is In Idle State, which is the correct answer to CMD0 and not an error.

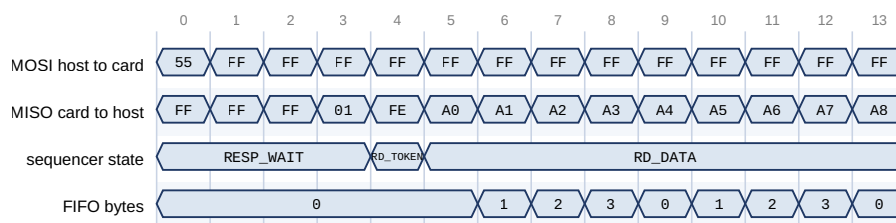
MISO reads 0xFF whenever the card is not driving it - the line is pulled up, and the host clocks 0xFF out to generate the clock.

Six bytes out — 0x40 | index, four argument bytes, then CRC7 shifted up one with the stop bit in bit 0 — and then the host clocks 0xFF until the card answers.

`N_CR` is a **range**, not a latency: the specification allows the response anywhere from 0 to 8 byte-times after the frame, so the core polls for a byte with bit 7 clear rather than waiting a fixed time. In the recording the card took two.

0x95 is the CRC7 of CMD0 with a zero argument, which is why that constant appears in every SD initialisation routine ever written.

6.3 The start of a block read



CMD17 READ_SINGLE_BLOCK, after the R1. The card may take as long as it likes before the block arrives, so the core sits in `RD_TOKEN` clocking 0xFF until it sees a token.

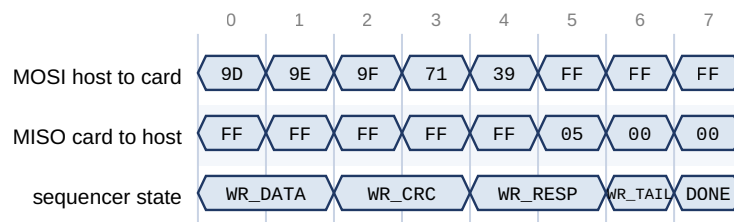
0xFE is the start-of-block token. Anything with the top four bits clear is a data error token instead, and the core reports it in `ERR_INFO` rather than treating the byte as data.

The 512 data bytes follow immediately, then a two-byte CRC16. The FIFO count rises as they land; the DMA drains it in parallel, which is why the count does not simply climb to 512.

After the R1 the card may take as long as it likes, so the core sits in `S_RD_TOKEN` clocking 0xFF until a token arrives. 0xFE starts a block; anything with the top four bits clear is a data error token instead, and is reported in `ERR_INFO` rather than treated as data.

The `FIFO bytes` row shows why the DMA matters: the count rises as bytes land and falls as the master drains them, in parallel, rather than climbing to 512 and then emptying.

6.4 The end of a block write



CMD24 WRITE_BLOCK, at the end of the block: the last data bytes, the two CRC16 bytes, then the card's data-response token.

The token is 0x05. Only five bits carry meaning - the shape is `xxx0sss1`, and `sss = 0b010` is "accepted".

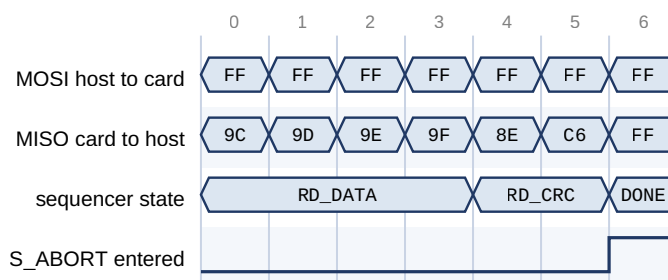
Comparing the whole byte against a constant is the classic way to get this wrong.

The card then held MISO low for 2 byte-times while it programmed the block. The core reports this as `CARD_BUSY` in `STATUS` and will not start the next command until it lifts.

The data-response token is the trap in the write path. Its shape is `xxx0sss1` — **five bits of meaning in eight** — and comparing the whole byte against a constant is the classic way to get this wrong. In the recording it is `0x05`: `sss = 010`, data accepted.

The card then holds `MISO` low while it programs the block. The core reports that as `CARD_BUSY` in `STATUS` and will not start the next command until it lifts.

6.5 A block whose CRC16 does not match



The same read as the previous figure, with the card corrupting the CRC16. The two CRC bytes arrive in `RD_CRC` exactly as they would if they were right - nothing on the wire says otherwise.

The core folds the final byte into the running CRC16 combinationally and checks the result against zero, so the mismatch is known at the end of the last CRC byte rather than a byte later. That is why the check is written that way.

`S_ABORT` is transient - it raises `dma_abort` and routes on to `S_DONE` within a cycle or two, which is why it never appears in the state row and needs a row of its own. `IRQ_STATUS` bit 12 (`ERR_DAT_CRC`) is set, and any DMA still in flight is drained rather than cut off mid-burst - an Avalon master that stops issuing beats mid-burst hangs the interconnect.

There is no `irq` row because it would be a flat 1: `DMA_DONE` has already raised the pin during the data phase. The pin says something happened; only `IRQ_STATUS` says what, which is why a handler must read it rather than infer the cause from the interrupt alone.

Nothing on the wire distinguishes a corrupt block from a good one until the check is done. The core folds the final byte into the running CRC16 *combinationally* and compares against zero, so the mismatch is known at the end of the last CRC byte rather than a byte later. Reading the register instead checks the CRC one byte early, which fails on every block, always, and looks exactly like a wiring or polynomial fault.

The `S_ABORT entered` row has to exist because `S_ABORT` is transient — it raises `dma_abort` and routes on to `S_DONE` within a cycle or two, so a byte-level state row never lands on it and the figure would otherwise say the error path was not taken.

Note also what the figure does *not* show. There is no `irq` row, because it would be a flat 1: `DMA_DONE` has already raised the pin during the data phase. The pin says something happened; only `IRQ_STATUS` says what.

Regenerating these figures

```
python3 doc/tools/diagrams/build_figures.py    # the block diagrams
./verification/capture.sh                     # record wave.vcd
python3 doc/tools/waveforms/mkwaves.py        # the timing figures
python3 doc/tools/check_facts.py              # re-derive every number above
```

`mkwaves.py` fails loudly rather than silently producing a wrong picture: if a scenario no longer contains the token or state it is looking for, it exits with an error naming what it could not find.

Revision history

Version	Change
v1.0	First issue. Five block diagrams, five timing figures generated from a recorded simulation.